Flamme

Build Plan

From prototype to evidence platform:
stack, architecture and step-by-step build

Team Flamme

NASA Space Apps Challenge · *Flame in Freefall*

October 2026

Guiding rule. Every value shown is either **Observed** (cited source row or page), **Predicted** (named model version with uncertainty) or **Unknown**. Nothing is ever silently filled in. Flamme is not a NASA product and must not be used for material certification or operational safety decisions.

Contents

1 What we are building **2**

2 The stack **3**

3 Repository layout (target) **4**

4 The core data model **4**
4.1 The ten conditions 5
4.2 Database tables 5

5 Step-by-step build **5**
5.1 Step 1 — Clean up the repository and set up tooling ($\frac{1}{2}$ week) 6
5.2 Step 2 — Define the schema in code ($\frac{1}{2}$ week) 6
5.3 Step 3 — Table ingestion pipeline ($1\frac{1}{2}$ weeks, deliverable 1) 6
5.4 Step 4 — Report extraction with human review ($1\frac{1}{2}$ weeks, deliverable 1) . . . 7
5.5 Step 5 — Search and compare (1 week, deliverable 2) 7
5.6 Step 6 — Gap map (1 week, deliverable 3) 7
5.7 Step 7 — Fit models to the data (2 weeks, deliverable 4: science) 8
5.8 Step 8 — Ground the 3D lab in data ($1\frac{1}{2}$ weeks, deliverable 4: visual) 8
5.9 Step 9 — Reporting template ($\frac{1}{2}$ week, deliverable 5) 9
5.10 Step 10 — Outreach mode ($\frac{1}{2}$ week, deliverable 6) 9
5.11 Step 11 — Quality, accessibility and deployment (1 week, overlapping) 9

6 Timeline **10**

7 Suggested team split **10**

8 Risks and mitigations **10**

Not a NASA product

1 What we are building

The plan document defines six deliverables on top of the existing prototype.

#	Deliverable	What it means technically
1	Evidence database	Every NASA experiment stored in the ten-condition schema, with a source link and a per-field "not reported" flag
2	Search and compare	Filter by any condition ("PMMA, $< 21\%$ O_2, opposed flow < 10 cm/s"); side-by-side view in standard units
3	Gap map	Coverage plot of tested versus untested condition combinations
4	Data-grounded 3D lab	Replace the hand-tuned curves in `labModel.js` with relationships fitted to the database; show the nearest real experiments beside the simulation
5	Reporting template	The ten-condition format published as a JSON Schema and a fillable template
6	Outreach mode	Keep the 92-second guided Simulate sweep for classrooms and the public

Reused from the current repository (not rewritten)

- `src/Lab.jsx`, `src/labScene.js`, `src/labModel.js` — the Three.js flame lab and 92-second timeline.
- `src/data.js`, `src/science.js` — the field model (`source / unit / normalized / status`) and transparent matching. This field model becomes the database schema.
- `flame_in_freefall_data/psi_scraper.py`, `download_ntrs.py` — PSI and NTRS harvesting (24 investigations, about 3,000 table rows, 36 PDFs).
- `findings/` — table-to-finding extraction and the PDF extraction utility.
- `parameter_catalog/` — the ten-condition reference.

2 The stack

Not a NASA product

Layer	Choice	Why
Frontend	React 19 + Vite 7 + TypeScript	Already used; TypeScript catches API/UI schema mismatches. Migrate gradually (`allowJs`)
3D	Three.js (existing `labScene.js`)	Already built; no rewrite needed
Routing / fetching	TanStack Router + TanStack Query	Typed, shareable search URLs; caching
Lab state	Zustand	Small; readable from the Three.js render loop
Tables	TanStack Table	Sortable, filterable compare view
Charts	Observable Plot (or ECharts)	Heatmaps and scatter plots with little code
Icons / style	Lucide, existing `app.css` as CSS variables	Keep the current look
API	Python 3.12 + FastAPI + Pydantic v2	Pipeline and ML are already Python; Pydantic generates the JSON Schema for deliverable 5
ORM / migrations	SQLAlchemy 2 + Alembic	Versioned schema changes
Database	PostgreSQL 16 on Neon or Supabase	Relational, range queries, JSONB for raw source rows
Units	`pint`	Safe atm/psi/kPa, %/mole-fraction, mm/cm, °C/K conversion
Wrangling	pandas (or polars) + pandera	Per-investigation adapters with schema checks
PDF extraction	PyMuPDF + pdfplumber + LLM structured output	Pull conditions from NTRS reports with page and quote; human review required
ML	scikit-learn, statsmodels, optional PyMC	Small data: simple, interpretable, calibrated models
Tracking	MLflow or versioned JSON in `models/`	Every fit has a version the UI can cite
Data versioning	DVC (Drive / S3 / R2 remote)	239 MB of raw data must not live in git
Auth	Supabase Auth (or Clerk)	Public site is read-only; only reviewers log in
Tests	Vitest, Playwright, pytest	Unit, browser and pipeline coverage
CI	GitHub Actions	Lint, test, build and data validation on every PR
Hosting	Vercel (web), Render/Fly-/Railway (API), Neon (DB)	Free tiers, push-to-deploy

Not a NASA product

Cheaper alternative for a hackathon window. Skip the live API. Run the pipeline at build time, export `experiments.json` and `models/*.json`, and query in the browser with **DuckDB-WASM**. Same schema, same pipeline, no server. Move to FastAPI + Postgres when the reviewer workflow (Step 4) is needed.

3 Repository layout (target)

```
flamme/
|-- apps/
|   |-- web/                  React + Vite + TS (moved from src/)
|   |   |-- src/lab/          Lab.tsx, labScene.ts, labModel.ts
|   |   |-- src/search/       filter bar, results, compare view
|   |   |-- src/gaps/         gap map
|   |   `-- src/api/          generated types + TanStack Query hooks
|   `-- api/                  FastAPI
|       |-- flamme_api/
|       |   |-- models.py     SQLAlchemy tables
|       |   |-- schemas.py    Pydantic: source of truth for the 10 conditions
|       |   |-- routes/       experiments, search, gaps, models, review
|       |   `-- query.py      filter -> SQL
|       `-- alembic/
|-- pipeline/
|   |-- harvest/              psi_scraper.py, download_ntrs.py (moved)
|   |-- adapters/             one per investigation: saffire2.py, bass2.py ...
|   |-- extract_pdf/          NTRS report extraction (LLM + quote + page)
|   |-- normalize.py          units via pint, vocabularies, status flags
|   |-- validate.py           pandera checks
|   `-- load.py               upsert into Postgres
|-- ml/
|   |-- features.py
|   |-- fit_flammability.py
|   |-- fit_spread.py
|   |-- fit_suppression.py
|   `-- export.py             -> models/<name>@<version>.json for the lab
|-- schema/
|   `-- flamme-experiment.schema.json    (generated; deliverable 5)
|-- data/                     DVC-tracked raw + processed
`-- docs/
```

4 The core data model

Build this first — everything else depends on it. Every condition value is a **Field**, exactly the shape already used in `src/data.js`:

```python
class Field(BaseModel):
    source: str | None          # verbatim: "~ 22.1", "037 mm", "Insignificant"
    value: float | str | None   # normalized, only when defensible
    unit: str | None            # standard unit (table below)
    status: Literal["reported", "approximate", "qualitative",
                    "ambiguous", "not_reported", "not_applicable", "derived"]
    note: str | None = None
```


Not a NASA product

4.1 The ten conditions

#	Condition	Fields (standard unit)
1	Material / fuel	name, material class (cellulosic, PMMA, silicone, fabric composite, liquid droplet, gas, …), formulation, coating, substrate
2	Geometry	length, width, thickness (mm); thermally thin/thick flag (derived)
3	Gravity	regime (Earth, lunar, Martian, microgravity), g level (g_0), platform (ISS, Cygnus, drop tower, parabolic)
4	Oxygen	X_{O_2} (% vol)
5	Pressure	P (kPa)
6	Forced airflow	u (cm/s)
7	Flow direction	concurrent, opposed, cross, quiescent
8	Ignition	igniter type, power (W), duration (s), position
9	Initial temperature	T_0 (K)
10	Suppression	agent, concentration (% vol), flow rate, timing (s after ignition)

Derived column. Oxygen partial pressure, because the plan states that it, not percentage alone, drives burning:

$$p_{O_2} = \frac{X_{O_2}}{100} P$$

It is stored with status `derived` and only when *both* inputs are reported.

4.2 Database tables

Table	Key columns
`investigation`	psi_id, name, platform, doi, url
`source_document`	id, investigation_id, kind (table / report / image), filename, url, sha256, version
`experiment`	id, investigation_id, source_document_id, source_row or page, quote, ten condition columns (each `value`, `unit`, `status`, `source`), raw_row JSONB
`outcome`	experiment_id, metric (spread_rate, burn_time, extinction, ignited, flame_length, …), value, unit, status, source
`review`	experiment_id, reviewer, decision (approved / corrected / rejected), diff JSONB, timestamp
`model_version`	name, version, trained_on (dataset hash), metrics JSONB, domain JSONB, created_at
`audit_log`	every normalization decision and correction — the auditable "agentic memory" from the README

Store condition columns *flat* (`oxygen_value`, `oxygen_status`, `oxygen_source`) rather than as one JSON blob. Flat columns make range search and the gap map fast and simple.

5 Step-by-step build

Each step ends with a **Done when** check. Sizing assumes a team of 2–4 people.

Not a NASA product

5.1 Step 1 — Clean up the repository and set up tooling ($\frac{1}{2}$ week)

1. Create `apps/web`, `apps/api`, `pipeline`, `ml`, `schema`; move files with `git mv` to keep history.
2. Web: add TypeScript (`allowJs: true`), Vitest, ESLint and Prettier.
3. API: `uv init apps/api`; add `fastapi uvicorn sqlalchemy alembic psycopg pydantic pint pandas pandera`.
4. `dvc init`; move `psi_data` and `ntrs_pdfs` into `data/raw`, `dvc add` it, configure a remote.
5. Add a `LICENSE` (the README notes there is none) and `.env.example`.
6. GitHub Actions: `web.yml` (lint, Vitest, build) and `api.yml` (ruff, pytest).

Done when: `npm run dev`, `npm test`, `uv run pytest` and CI all pass on a clean clone; the existing five science tests stay green.

5.2 Step 2 — Define the schema in code ($\frac{1}{2}$ week)

1. Write `schemas.py` (Pydantic) for `Field`, the ten conditions, `Outcome` and `Experiment`.
2. Write SQLAlchemy models and the first Alembic migration.
3. Generate `schema/flamme-experiment.schema.json` with `Experiment.model_json_schema()`.
4. Generate TypeScript types from the OpenAPI spec with `openapi-typescript`.
5. Port the five curated records in `src/data.js` into a seed file. They are the golden test set.

Done when: the five seed records round-trip DB → API → UI unchanged, including `037 mm` staying `ambiguous` and pressure staying `not_reported`.

5.3 Step 3 — Table ingestion pipeline ($1\frac{1}{2}$ weeks, deliverable 1)

PSI experimental tables all have different columns, so write **one adapter per investigation**, not one generic parser.

1. For each of the 22 table files, write `pipeline/adapters/<investigation>.py` with a `COLUMN_MAP` (source column → condition) and a `parse_row()` that keeps `source` verbatim.
2. `normalize.py`: convert units with pint; map vocabularies ("co-current", "cocurrent" → `concurrent`); set status (`~` → approximate, words → qualitative, unparseable → ambiguous, blank → not_reported).
3. `validate.py` (pandera): $0 \le X_{O_2} \le 100$, $0 \le P \le 200$ kPa, $u \ge 0$, thickness > 0, and every row has a source file and row number.
4. `load.py`: idempotent upsert keyed on (source document, source row).
5. Lift the parsing logic already in `findings/build_findings.py` into the adapters.

Priority: the investigations the plan names — **Saffire** (PSI-99 etc.), **BASS / BASS-II** (PSI-25), **FLEX** (PSI-69) — then the rest.

Done when: `make ingest` loads every table investigation; each adapter has a pytest checking row counts and three hand-verified rows; a validation report shows the "not reported" rate per field.

Not a NASA product

5.4 Step 4 — Report extraction with human review (1½ weeks, deliverable 1)

Many conditions (pressure, ignition energy, flow direction) appear only in NTRS reports, drop-tower and parabolic-flight papers, and NASA-STD-6001 test reports.

1. `extract_pdf/`: PyMuPDF for text, pdfplumber for tables; chunk by page.
2. Ask an LLM for **structured output** matching the Pydantic `Experiment` schema, with a `page` and verbatim `quote` for every non-null field. Reject any field whose quote is not found in the page text (string match in code).
3. Everything extracted lands as `pending`; nothing pending is shown publicly.
4. Build a `/review` screen (reviewer login): PDF page on the left, extracted fields on the right; approve, correct or reject. Corrections go to `audit_log`.

Done when: all 36 downloaded NTRS PDFs are processed; at least one reviewer has approved records from each; a test confirms unapproved rows never appear in public API responses.

5.5 Step 5 — Search and compare (1 week, deliverable 2)

1. API: `GET /experiments?material_class=PMMA&o2_lt=21&direction=opposed&flow_lt=10`; each filter maps to SQL on the flat columns.
2. Show how unknowns behave: a row with oxygen `not_reported` is excluded from an O_2 filter but counted in a "12 more records have unknown O_2" link.
3. Free-text bar: a deterministic parser (regex + vocabulary) turns the plan's example query into a filter object shown back as editable chips. An optional LLM fallback may *propose* filters; the user always sees the chips first.
4. Keep the `science.js` similarity ranking as "closest experiments", now server-side over the whole database.
5. Compare view: up to four experiments as columns, the ten conditions as rows, standard units, unknown cells visibly marked, each cell linked to its source row or page.
6. Filters live in the URL so searches can be shared.

Done when: the plan's example query returns the right rows; a Playwright test covers search → compare → open source link.

5.6 Step 6 — Gap map (1 week, deliverable 3)

1. API `GET /gaps?x=oxygen&y=flow&facet=gravity` returns counts per bin. Default bins: O_2 2 percentage points, flow 5 cm/s, pressure 10 kPa.
2. Heatmap: coloured cells are tested (count shown), outlined empty cells are **gaps**, hatched cells contain only approximate values.
3. Clicking a cell opens the search results for that bin. Facets: gravity, flow direction, material class.
4. Overlay the fitted flammability boundary from Step 7, so users see conditions that are untested *and* near the boundary — the most useful place for a next experiment.

Not a NASA product

Done when: map counts match a direct SQL query (pytest), and clicking any tested cell returns exactly that many rows.

5.7 Step 7 — Fit models to the data

(2 weeks, deliverable 4: science)

Data is small and heterogeneous, so use simple, physics-informed, interpretable models — one per question, never one model for everything.

Model	Inputs	Target	Method
Flammability boundary	p_{O_2}, u, material class, gravity	ignited and sustained (yes/no)	Logistic regression or Gaussian-process classifier
Spread rate	thickness, p_{O_2}, u, direction	spread rate (mm/s), only where sources measure the same quantity	Physics-form regression
Suppression	agent, concentration, u	extinguished (yes/no), time to extinction	Logistic regression or survival model

For spread over a **thermally thin** sample in opposed flow, use the de Ris result as the model shape instead of a black box:

$$V_f = \frac{\pi}{4} \frac{\lambda_g (T_f - T_v)}{\rho_s c_s \tau (T_v - T_\infty)}$$

so spread rate should scale as $V_f \propto \frac{1}{\tau}$ with sample thickness τ. Fit the coefficient (and any extra oxygen and flow dependence) from data, and report how well the observations follow this shape.

Rules for every fit

1. **Grouped cross-validation by investigation** (`GroupKFold`) so one experiment never sits in both train and test.
2. Start with a baseline (mean or single variable); keep a model only if it beats it.
3. Report calibrated uncertainty (prediction intervals or calibrated probabilities).
4. Record the **domain** (ranges or convex hull of training inputs). Outside it the UI says "outside tested conditions" instead of predicting.
5. `ml/export.py` writes `models/flammability@1.0.json` (coefficients, domain, metrics, dataset hash) and registers it in `model_version`.

Done when: each model card shows cross-validated metrics against the baseline, the domain and the dataset version; a pytest confirms export → reload gives identical predictions.

5.8 Step 8 — Ground the 3D lab in data

(1½ weeks, deliverable 4: visual)

1. Split `derive(p)` in `labModel.js` into two layers:
   - `predict(p)`: physical quantities from the exported model JSON (ignition probability, spread rate with interval, extinguished or not);

Not a NASA product

- `toVisual(prediction)`: flame size, soot, shell shape and colour, keeping the existing Three.js look.

2. Load models with `fetch('/models/flammability@1.0.json')`; logistic and linear models run in a few lines of JavaScript in the browser.
3. Out of domain: keep the illustrative curve and show the banner *"Illustrative — no NASA data for these conditions"*.
4. Add a **Closest real experiments** panel: query the similarity endpoint with the current slider values and show the top three with real outcome and source link.
5. Label every on-screen number **Observed**, **Predicted (model@version)** or **Unknown**.

Done when: for each seed experiment, setting the sliders to its conditions shows it as the closest experiment, and the predicted outcome direction agrees with the observed one — or the mismatch is shown, not hidden.

5.9 Step 9 — Reporting template

($\frac{1}{2}$ week, deliverable 5)

1. Publish `schema/flamme-experiment.schema.json` (generated in Step 2) at `/schema`.
2. Generate a fillable CSV template and a web form from the same schema (`react-jsonschema-form`) that flags missing conditions.
3. A "Validate my file" page: upload CSV or JSON, receive a completeness report for the ten conditions.

Done when: a file exported from the database validates against the schema; a file missing pressure gets a clear warning.

5.10 Step 10 — Outreach mode

($\frac{1}{2}$ week, deliverable 6)

1. Keep the 92-second `/#studio-play` sweep; captions now cite real experiments ("Saffire II sample 2-5 at about 22% O_2 …").
2. A classroom route with large text and no admin UI, available offline as a PWA (`vite-plugin-pwa`).

Done when: the sweep runs end to end on a school laptop in Chrome or Firefox at 30 fps or more.

5.11 Step 11 — Quality, accessibility and deployment

(1 week, overlapping)

1. Tests: Vitest for `labModel` and `science`; Playwright for search → compare → lab → source; pytest for adapters, API and model export; a visual snapshot of the lab.
2. Accessibility: keep the keyboard shortcuts, add `aria-live` for lab status, audit with axe.
3. Performance: lazy-load the Three.js lab chunk; cap device pixel ratio at 2.
4. Deploy: web to Vercel (`VITE_API_URL`); API to Render or Fly (`DATABASE_URL`); database on Neon; a release job runs `alembic upgrade head` and `make ingest`.
5. Footer on every page: "Not a NASA product. Not for certification or operational safety decisions."

6 Timeline

Week	Steps	Milestone
1	1, 2	Tooling, schema, five seed records through the full stack
2–3	3	All PSI tables in the database
3–4	4, 5	Reports extracted and reviewed; search and compare live
5	6, 9	Gap map; reporting template
6–7	7	Fitted models with model cards
7–8	8, 10, 11	Data-grounded lab, outreach mode, deployment

48-hour hackathon path. Steps 1–3 for Saffire, BASS and FLEX only; Steps 5 and 6 with DuckDB-WASM over exported JSON; one flammability-boundary model for Steps 7–8; and the closest-experiments panel. That alone demonstrates the full idea end to end with real data.

7 Suggested team split

Role	Owns
Data	Steps 3 and 4: adapters, normalization, PDF extraction, review
Backend	Step 2, the APIs for Steps 5 and 6, deployment
Frontend / 3D	The interfaces for Steps 5 and 6, plus Steps 8 and 10
ML / science	Step 7, model cards, checking physics and limitations

8 Risks and mitigations

Risk	Mitigation
Too few comparable outcome measurements for a model	Fit per question and per material class; if data is too thin, show only nearest experiments with “insufficient data”
LLM extraction invents values	Page and quote required, quote string-matched in code, human approval before publishing
Unit or definition mismatch (e.g. Saffire “Spread Length” column holding mm/s)	Keep `source` verbatim, mark `ambiguous`, exclude from fits until reviewed
Leakage inflating model scores	`GroupKFold` by investigation
Users treat the lab as a real fire prediction	Observed / Predicted / Unknown labels, out-of-domain banner, disclaimer footer
Raw data size (about 14 TB on PSI)	Tables and reports only; raw downloads stay opt-in and size-capped

Not a NASA product